

Chapter 9, Before You Read

Processes and scheduling

Chapter 9 covers what an operating system is actually doing underneath your programs. This reading covers processes and scheduling: what services an OS provides, how early systems worked before an OS existed at all, and how the OS tracks and schedules the processes it is juggling.

1. Why This Chapter Matters for Your Distribution

Of everything in this book, this chapter is the one that maps most directly onto what you are building. A kernel has to decide which process runs next, and it has to decide what happens to a process when it is waiting on something. Those two decisions are the entire subject of this chapter. When you configure a scheduler, write an init system, or debug a process that will not die, you are working with the exact ideas below, not an abstraction of them.

2. What an Operating System Actually Does

An operating system is the most important program on the machine. It hides the details of the hardware from everything running above it and gives every program a consistent way to ask for what it needs. In broad terms, it provides services in a handful of areas: creating programs, running them, reaching I/O devices, controlling access to files, controlling access to the system itself, detecting and responding to errors, and keeping usage accounting.

It helps to name three interfaces this involves, because you will meet all three again in your own project.

- **ISA**, the instruction set architecture, is the boundary between hardware and software: the instructions a processor actually understands. An application gets a restricted subset of that instruction set. The OS gets a wider, privileged subset, which is exactly what lets it do things a user program cannot.
- **ABI**, the application binary interface, is what makes a compiled binary portable across systems that share it. It defines the system call interface along with what the user ISA exposes.
- **API**, the application programming interface, is the ABI plus library calls at a higher level. Source code recompiled against the same API can move between systems, even when the binary cannot.

The operating system is also unusual as a piece of software. It runs the same way any other program runs, fetched and executed by the processor, yet it constantly gives up control and depends on the processor handing that control back later. Nearly everything else in this chapter is really about how that handoff is managed well.

3. From No Operating System to a Resident Monitor

It is worth knowing what things looked like before an OS existed, because the problems that early setup produced are the same problems every design choice below is solving.

From the late 1940s into the mid 1950s, a programmer worked the machine directly from its console: switches, lights, a card reader, a printer. Two problems came out of that arrangement. Scheduling was handled with a sign up sheet, so a job that finished early wasted machine time and a job that ran long got cut off. Setup time was also large, since loading a compiler, loading a program, saving the result, and linking everything together all had to happen by hand for every single job.

The fix was the resident monitor, a small program that stayed in memory at all times and read in jobs one after another with no idle gap between them. The user no longer touched the hardware directly. Instead, a job was submitted with a short set of job control instructions attached to it, telling the monitor which compiler to load and when to run, so the monitor could move from job to job without a human in the loop.

Making this work reliably needed a few things from the hardware itself, and these are worth sitting with, because they are the same protections your own kernel work depends on.

- **Privileged instructions.** Only the monitor may execute them. If a user program tries, the hardware raises an error interrupt. I/O instructions are privileged for exactly this reason, so a user program can never read another job's data directly off a device.
- **Memory protection.** A user program is not allowed to alter the memory holding the monitor. The hardware, not the monitor itself, is what catches the attempt and hands control back.
- **A timer.** Prevents one job from holding the processor forever. When it expires, an interrupt fires and control returns to the monitor.
- **Interrupts.** Give the OS a reliable way to hand control to a user program and get it back, rather than hoping the program eventually returns control on its own.

4. Multiprogramming and Time Sharing

A single resident monitor running one job at a time still leaves the processor idle most of the time, because I/O is slow compared to computation. Multiprogramming is the fix: keep more than one program in memory, and when one is waiting on I/O, switch the processor to a program that is not. Extending this idea to interactive users rather than batch jobs is called time sharing, where the processor's time is divided into short quanta and shared out among everyone connected at once. With n users actively requesting service, each one sees roughly $1/n$ of the machine's effective speed, before accounting for OS overhead.

Table 1: Batch multiprogramming compared with time sharing.

	Batch Multiprogramming	Time Sharing
Principal objective	Maximize processor use	Minimize response time
Directives to the OS	Job control language, with the job	Commands typed at the terminal

5. Processes, State, and Scheduling

The word process is the more general replacement for job once multiprogramming is in the picture. At any moment, a process is in one of five states.

- **New.** Admitted, but not yet ready to run.
- **Ready.** Waiting only for access to the processor.
- **Running.** Actually being executed right now.
- **Waiting.** Suspended until some resource, usually I/O, becomes available.
- **Halted.** Finished, and about to be cleaned up by the OS.

For every process, the OS keeps a process control block, a small record that holds an identifier, the current state, a priority, the program counter, where the process lives in memory, the saved register contents from the last time it ran, its open I/O requests, and accounting information such as CPU time used. This is what lets a process stop running and later resume exactly where it left off. If you have looked at `/proc/<pid>/status` on a running Linux system, you have already seen a real process control block, just exposed as readable text.

Deciding which process gets the processor, and when, happens at three levels, each operating on a different timescale.

- **Long term scheduling** decides whether a new process is admitted at all, and controls how many processes are competing for memory at once.
- **Short term scheduling**, also called the dispatcher, runs constantly and makes the fine grained decision of exactly which ready process executes next.
- **Medium term scheduling** sits between the two, and is part of the swapping mechanism described below: it manages the degree of multiprogramming by deciding which processes are temporarily moved out of memory.

A fourth kind of scheduling, I/O scheduling, decides which of several pending I/O requests an available device handles next.

6. Swapping, a Preview of What Comes Next

Even with multiprogramming, it is possible for every resident process to end up waiting on I/O at the same time, at which point the processor sits idle again despite all the machinery already built to prevent exactly that.

Swapping addresses this by physically relocating a process rather than merely tracking its state. When the short term scheduler looks at the ready queue and finds it empty, because everything currently in memory is blocked on I/O, the medium term scheduler steps in. It selects one of those waiting processes and writes its entire memory image, code, data, and stack, exactly as they currently sit in RAM, out to a reserved area of disk. That frees the memory the process was occupying for something that can actually use the processor right now: either a new process admitted from the long term queue, or a process that was swapped out earlier and is now ready to resume. Processes waiting their turn to come back into memory sit in an intermediate queue, which is a different thing from the long term queue: a process in the intermediate queue has already

run before and only needs memory again, while a process in the long term queue has never been admitted at all.

Concretely, one full cycle looks like this.

- The scheduler notices every resident process is currently waiting on I/O, so the processor has nothing ready to run.
- It selects one of those waiting processes and writes its entire memory image out to a reserved area of disk, exactly as it currently sits in RAM.
- The memory that process was occupying is now free, so the OS loads in a different process, either a new one from the long term queue or one that was swapped out earlier and is now ready.
- The process that was swapped out sits in the intermediate queue on disk until the OS brings its memory image back in, at some later point when it is again its turn and space allows.

It is worth sitting with how expensive this actually is. A process's memory image can run to many megabytes, and writing all of it to disk and later reading all of it back is a real, measurable amount of time, not a free bookkeeping trick. The OS specifically targets a waiting process for this rather than a ready one, since a waiting process cannot use the processor anyway, so moving it out of memory costs nothing in missed opportunity, while pulling out a ready process would mean giving up a process that could have run immediately. Swapping is also, itself, an I/O operation, which sounds self defeating given that I/O congestion is the entire reason multiprogramming exists. It works out only because disk I/O is generally the fastest I/O available on the machine, so the OS is trading a wait on a slow device for a comparatively fast round trip to disk.

That speed relationship is also exactly why swapping is not the final answer. Moving an entire process's memory image, all at once, whether or not the process actually needs all of it soon, is expensive every single time. That is where the next part of this chapter picks up: partitioning, paging, and virtual memory let the OS move much smaller, fixed size pieces of a process instead of the whole thing, and only the pieces actually in use, which is the idea that decides how much of physical memory your own kernel actually has to work with at any moment.